

High-level rules for the plan

- Use **shared DB with organization_id** on all tenantable tables (enforced from Day 1).
 - Start with a **modular monolith** backend (packages/modules). Later you can split into microservices.
 - Frontend: React (Create React App / Vite / Next.js) with component-first approach. Use a simple auth flow to begin (JWT stored in memory + refresh token handling).
 - Sample stack (recommended): Spring Boot, Spring Security (JWT), Spring Data JPA (Hibernate), MySQL, Redis (cache), MinIO/S3 (files). Frontend React + Tailwind (or MUI) + React Router + state management (Redux / Zustand).
-

Phase 0 — Repo & infra scaffolding (pre-work, mandatory)

Why first: Create a reproducible dev environment and CI so you can iterate and deliver demos consistently.

Backend

- Initialize Git repo and branches (main, develop, feature/*).
- Create Spring Boot project skeleton with modules (core, auth, org, user, team, project, task, client, file, comms, audit).
- Add basic dependencies: Spring Web, Spring Data JPA, Spring Security, Lombok, Flyway/Flyway or Liquibase, MapStruct, SpringDoc.
- Add BaseEntity, Auditing, ApiResponse, global exception handler.
- Setup application profiles: dev, test, prod.
- Setup local MySQL docker compose, Redis, MinIO (optional) in docker-compose.yml.
- Add basic logging and formatter; enable SQL logging for dev.
- Setup CI pipeline skeleton (GitHub Actions): build, test, lint.

Frontend

- Initialize frontend repo (mono or in same repo as subfolder).
- Create app skeleton (routing, layout, login page placeholder).
- Add component library and CSS system (Tailwind/MUI) and basic auth context/provider.
- Setup dev proxy to backend (so frontend can call local backend without CORS pain).
- Add basic ESLint/prettier config, and CI step to run lint/tests.

Deliverable checkpoint

- Full dev environment bootstraps with docker-compose up and both apps run locally.
-

Phase 1 — Core tenancy + Authentication & Authorization (foundational)

Why first: Every endpoint and UI needs a secure, tenant-scoped context and dynamic RBAC.

Backend (must)

1. Entities & persistence:

- Organization entity.
- User / AuthUser entity (with organization_id link), password (bcrypt), status.
- Role, Permission, UserRole, RolePermission entities (organization-scoped).
- BaseEntity with organization_id property.

2. Tenant context:

- OrganizationContextHolder (thread-local), TenantInterceptor to extract organization_id from JWT.
- Hibernate Filter / JPA Specification to enforce organization_id on queries.

3. Auth endpoints:

- POST /api/auth/login → returns JWT + refresh token (JWT contains userId, orgId, roles).
- POST /api/auth/refresh → refresh JWT.
- POST /api/auth/register (org creation + admin user) — admin bootstrap.

4. Role/Permission admin endpoints:

- CRUD for Roles: POST /api/orgs/{orgId}/roles
- CRUD for Permissions: POST /api/orgs/{orgId}/permissions
- Assign user to role: POST /api/orgs/{orgId}/users/{userId}/roles

5. Authorization framework:

- `AuthorizationService.can(user, action, resourceType, resourceId)` (skeleton).
- Custom annotation `@RequiresPermission(resource, action)` + AOP interceptor that uses `AuthorizationService`. Note: annotation metadata is generic and resolved dynamically.

6. Tests:

- Unit tests for `AuthService`, `UserRepository`.
- Integration test: login → call protected endpoint, validate 401/200.

Frontend (parallel)

1. Auth flow:

- Build Login page + register org/admin page.
- Implement Auth context/provider (store token in memory/localStorage per you).
- Implement refresh token handling.

2. Role admin UI:

- Minimal Role management UI (list/create roles, attach permissions).

3. Tenant workflow:

- When logged in, show `orgId` in header to verify tenant context.

4. Simple protected route guard (reads JWT, redirects to login if missing).

Deliverable checkpoint

- Can create org + admin user, login, JWT returned with org claim. Protected backend endpoints enforce tenant scoping. Frontend login + role UI works.

Phase 2 — Employee & Team Management (core people)

Why next: Teams & employees are used by Projects, Time tracking, Tasks, and Client mapping.

Backend

1. Entities:

- Employee (linked to User optionally), fields: employeeNumber, name, email, designation, joiningDate, status, managerId.
- Department.
- Team, TeamMember (team_id, user_id, role_in_team).

2. APIs:

- Employee CRUD: POST /api/orgs/{orgId}/employees, GET /api/orgs/{orgId}/employees/{id}
- Department CRUD.
- Team CRUD and membership management: POST /api/orgs/{orgId}/teams, POST /api/orgs/{orgId}/teams/{id}/members

3. Business rules:

- On employee create, ensure organization_id set from tenant context.
- Manager/lead validation: reference belongs to same org.

4. Tests:

- Integration tests for tenant isolation: create employee in Org A, ensure Org B cannot see.

Frontend

1. Employee pages:

- Employee list, create/edit form, profile page (with onboarding docs upload link).

2. Team pages:

- Team list, create team, add members modal.

3. Role-based rendering:

- Only org-admin/HR role sees employee admin UI.

Deliverable checkpoint

- Create employees, teams, assign members, and view within same org in both backend and frontend.
-

Phase 3 — Client Management (client portal groundwork)

Why now: Projects will attach to clients; clients can have users who log in.

Backend

1. Entities:

- ClientOrganization (client_company) with contact persons, address, status.
- ClientUser (client users; optionally a separate auth table or shared User with userType).
- ClientProjectAccess mapping (which client has access to which projects).

2. APIs:

- Client onboarding: POST /api/orgs/{orgId}/clients
- Client user invitation: POST /api/orgs/{orgId}/clients/{clientId}/invite
- Client user accept/complete registration flow (signup with invite token).

3. Auth:

- If using same User table, differentiate user_type (employee vs client_user) and enforce permission defaults.

4. Policies:

- Default client role & permission templates (view/project-comment).

5. Tests:

- Ensure client user can only access their client's projects.

Frontend

1. Admin UI:

- Create client org, invite client user

2. Client portal (basic):

- Separate route /client with login flow and dashboard (simple projects list).

3. Messaging:

- Simple client-team message thread UI placeholder.

Deliverable checkpoint

- Admin invites client, client user accepts, logs in, sees assigned projects list.
-

Phase 4 — Project Management (projects + teams + clients)

Why now: Projects are core and used by tasks, bugs, time logs, files.

Backend

1. Entities:

- Project with organization_id, client_id (nullable), visibility (private/team-scoped), start/end date, status, config.
- ProjectTeam (project_id <-> team_id)
- ProjectPhase, Milestone (optional)

2. APIs:

- POST /api/orgs/{orgId}/projects
- GET /api/orgs/{orgId}/projects/{id}
- Associate teams: POST /api/orgs/{orgId}/projects/{id}/teams

3. Business logic:

- Project visibility enforcement (client-specific vs org-internal).
- Project-level roles (project manager).

4. Tests:

- Project creation, access control, client visibility.

Frontend

1. Project UI:

- Create project wizard (link to client, teams).
- Project detail page (overview, assigned teams, milestones).

2. Kanban placeholder (board column UI — no drag yet).

3. Project access for client users: Projects they are authorized to view.

Deliverable checkpoint

- Create and view projects, assign teams and clients, verify access in frontend and backend.
-

Phase 5 — Task Management & Kanban basics

Why now: Tasks depend on Projects and Users/Teams.

Backend

1. Entities:

- Task (project_id, title, description, assignee_id, reporter_id, status, priority, estimate, labels)
- TaskAssignment and TaskComment entities
- TaskStatus and TaskColumn for Kanban

2. APIs:

- CRUD: POST /api/orgs/{orgId}/projects/{pid}/tasks
- Move task between columns: PATCH /api/orgs/{orgId}/tasks/{id}/move
- Comment on task: POST /api/orgs/{orgId}/tasks/{id}/comments

3. Business rules:

- Only users with task:create permission in that project/team scope can create tasks.

4. Tests:

- Authorization tests: ensure RBAC works for creating/updating tasks.

Frontend

1. Task components:

- Task creation modal, task detail panel, comments.

2. Kanban board:

- Render columns and tasks; implement drag & drop later.

3. Notifications UI for assignment/comment.

Deliverable checkpoint

- Create tasks, comment, and view in Kanban-like board; permissions enforced.

Phase 6 — Bug Tracking & Issue Lifecycle

Why now: Bugs are similar to tasks but need priority/workflow.

Backend

1. Entities:

- Issue with type (TASK/BUG/FEATURE), severity, reproSteps, environment
- Link Issue to Task optionally

2. APIs:

- Issue create: POST /api/orgs/{orgId}/issues
- Assign, change status, link to project/task

3. Tests:

- Issue flow tests (create → assign → close)

Frontend

1. Issue forms and list filter (by severity/status)
2. Issue detail + link to task

Deliverable checkpoint

- Report bugs (internal/client), workflow visible and functional.

Phase 7 — Time Tracking & Reports

Why now: Time entries need tasks/projects and user data.

Backend

1. Entities:

- TimeEntry (user_id, task_id, start_time, end_time, duration, billable)

2. APIs:

- POST /api/orgs/{orgId}/time-entries
- GET /api/orgs/{orgId}/reports/time?from=&to=&projectId=

3. Business logic:

- Validate overlapping entries, enforce working hours defaults if needed.

4. Tests:

- Time entry creation, report aggregation.

Frontend

1. Time tracking UI:

- Start/Stop timer on task, manual entry form.

2. Reports page:

- Show time by user, project, week/month.

Deliverable checkpoint

- Track time against tasks; generate simple reports.

Phase 8 — File Management & Sharing

Why now: Files attach to tasks/projects and client communications.

Backend

1. Entities:

- FileResource (owner, project_id, task_id, path, metadata, version)

2. Storage:

- Integrate MinIO or S3-compatible storage.
- Signed URL generation for secure download.

3. APIs:

- Upload: POST /api/orgs/{orgId}/files (multipart)
- Download: GET /api/orgs/{orgId}/files/{id}/download
- Access control: enforce by org + project/team role

4. Tests:

- File upload/download flow + permission checks.

Frontend

1. File upload UI in Project/Task pages and Client portal
2. File list with preview/download actions

Deliverable checkpoint

- Upload and share files securely with role-based visibility.

Phase 9 — Communication & Notifications

Why now: Messaging ties client/team workflows together.

Backend

1. Entities:

- MessageThread, Message, Notification

2. Realtime:

- Implement WebSocket (Spring Websocket / STOMP) or fallback polling.

3. APIs:

- Thread create: POST /api/orgs/{orgId}/threads
- Send message: POST /api/orgs/{orgId}/threads/{id}/messages
- Notification push endpoints and unread counters.

4. Tests:

- Message delivery, thread visibility for client vs internal.

Frontend

1. Chat UI:

- Threaded message view per project/team/client.

2. Notifications:

- Bell icon with unread counts; list view.

Deliverable checkpoint

- In-app messaging between team members and clients; notifications emitted.
-

Phase 10 — Meetings & Calendar Integration

Why now: Meetings require users, projects, and clients set up.

Backend

1. Entities:

- Meeting, MeetingParticipant, MeetingMinutes

2. APIs:

- Create meeting: POST /api/orgs/{orgId}/meetings
- Invite participants (employees & clients)

3. Integrations:

- Optional calendar integration hooks (Google/Outlook) later.

4. Tests:

- Meeting creation and invites.

Frontend

1. Meeting scheduler UI with attendee selection

2. Meeting detail view and minutes editor

Deliverable checkpoint

- Schedule meetings and record minutes; participants notified.
-

Phase 11 — Dashboards, Reporting & Analytics

Why now: After modules exist, dashboards empower users.

Backend

1. Dashboard engine:
 - Build saved queries + widget model.
 - Expose analytics endpoints for time, task status, client health.
2. Data:
 - Aggregation endpoints; consider caching in Redis.
3. Tests:
 - Query correctness and security.

Frontend

1. Dashboard pages per role (Admin, Manager, Employee, Client)
2. Widgets for KPIs: open tasks, burn-down, time spent, project health.

Deliverable checkpoint

- Role-based dashboards rendering live data.
-

Phase 12 — Dynamic Policy Engine & ABAC/OPA integration

Why now: Policies require all resources and attributes available.

Backend

1. Policy storage:
 - Policy table storing JSON/OPA rules (org-scoped).
2. PDP:
 - Implement a policy evaluator (SpEL evaluator or integrate OPA).
 - Expose `/api/orgs/{orgId}/authorize` for resource-based checks.
3. Replace or augment `AuthorizationService` to consult policies.

4. Tests:

- Policy unit tests and scenario tests (simulate user actions).

Frontend

1. Admin UI for policy creation and simulator (test user + action + resource → decision).
2. Integration:
 - Optionally show policy reasons in UI (why access denied).

Deliverable checkpoint

- Dynamic policy authoring + runtime evaluation; no hardcoded permission checks.
-

Phase 13 — AI Features & Automation (phase 2+)

Why now: Use data from modules for analytics and automation.

Backend

1. Integrations:
 - LLM connectors for summarization (meetings, weekly report).
 - ML model endpoints for risk prediction or auto-assignment.
2. APIs:
 - POST /api/orgs/{orgId}/ai/summarize
 - POST /api/orgs/{orgId}/ai/triage
3. Tests:
 - Safe-guard prompt chains and cost-control.

Frontend

1. AI features in UI:
 - “Summarize meeting” button
 - Smart-assistant modal for creating tasks via NL
2. Usage controls:
 - Admin toggles for enabling AI per org.

Deliverable checkpoint

- LLM-powered helpers integrated with governance and admin toggles.
-

Cross-cutting tasks (done iteratively throughout)

- **Logging & Observability:** Setup structured logs, integrate OpenTelemetry; wire metrics to Prometheus/Grafana.
 - **Security & SAST:** Run dependency checks and static security scans; implement rate limiting and CORS policy.
 - **Testing:** Unit tests, integration tests, end-to-end tests (Cypress) for critical flows.
 - **CI/CD:** Build → test → deploy to dev/staging; DB migration steps (Flyway).
 - **Documentation:** API docs (OpenAPI/Swagger), developer README, and frontend storybook for UI components.
 - **Data migration tooling:** simple CLI or migration scripts if you import old data.
-

Acceptance criteria per module (what “done” looks like)

For every phase, ensure:

1. Backend: Entities migrated + DB migration script exists; APIs documented in Swagger.
 2. Frontend: Pages/components implement CRUD for that resource and show tenant-scoped data.
 3. Security: All API calls are authenticated and tenant-scoped; unit/integration tests assert this.
 4. Demo: Feature demoable end-to-end (login → create resource → client/employee access).
 5. CI: Tests run and pass on PR; build artifacts generated.
-

Suggested Development Workflow & Priorities

- Always implement **backend API contract** first for the feature (endpoint signatures + request/response). Then implement frontend to consume it.
- Write **integration tests** for backend endpoints as you implement them.

- Keep UI minimal and functional at first; polish later.
 - Use feature flags for big features (policy engine, AI).
 - Maintain a **permission simulator** UI early (helps debug RBAC problems).
-

First actionable checklist (start immediately)

1. Bootstrap repo, docker-compose, skeleton backend + frontend.
2. Implement Phase 1 entirely (Org + Auth + RBAC + tenant filter) — this is blocker for everything else.
3. Implement Phase 2 (Employee & Team) + frontend pages.
4. Implement Phase 3 (Client) + client portal skeleton.
5. Move forward phase-by-phase as above.